MAQ Software

C# OOPS Concepts

Friday, October 9, 2026

Headquarters:

2027 152nd Avenue NE
Redmond, WA 98052

Microsoft Premier Supplier

Gold Microsoft Partner

Winner Microsoft Partner 2021 Partner of the Year Power BI Award

amazon web services Partner Network CONSULTING PARTNER

bsi. ISO/IEC 27001 Information Security Management

Inc. 5000 The Fastest Growing Private Companies in America

Agenda

- Object and Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

OOP

OOP

- Object Oriented Programming (OOP) is a programming model where programs are organized around **objects** and **data**.
- OOP allows **decomposition** of a problem into several entities called **objects** and then builds **data** and **functions** around these objects.
- A class is the **core** of any modern Object-Oriented Programming language such as C#.
- In OOP language, it is mandatory to create a class for representing data.

Object and Class

Class in C#

- A class is a **blueprint** of an object.
- It defines **fields (variables), methods (functions), properties, constructors, events, etc.**
- A class will **not occupy** any **memory** space and hence it is only a logical representation of data.
- To create a class, use the keyword "**class**" followed by the class name:

```
class Car
{


}
```

Class in C#

```
public class Car
{
   // Fields (variables)
   public string brand;
   public int speed;

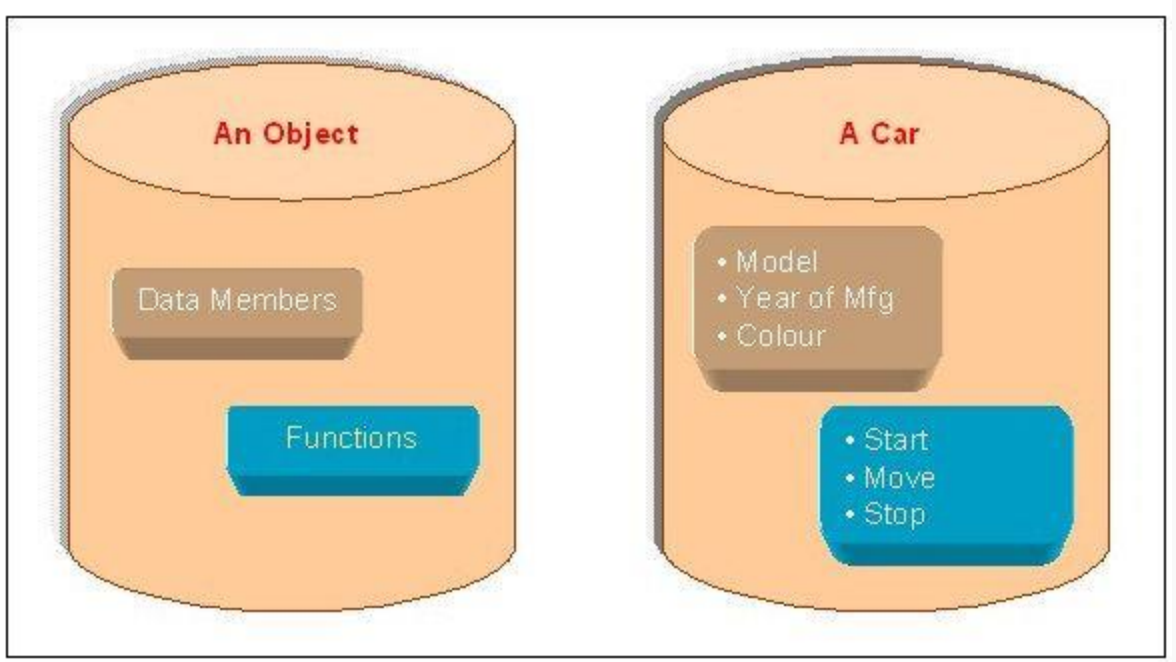
   // Constructor
   public Car(string brandName, int carSpeed)
   {
      brand = brandName;
      speed = carSpeed;
    }

    // Method
    public void Drive()
    {
      Console.WriteLine(brand +"is driving at" + speed + "km/h");
    }
}
```

Object in C#

- Objects are the basic **run-time** entities of an object-oriented system. They may represent a person, a place or any item that the program must handle.
- An object is a bundle of **related** variable and methods.
- An object is an **instance** of a class

```
Car obj = new Car();
```



Objects in C#

```
class Program
{
    static void Main(string[] args)
    {
        // Creating objects of Car class and invoking constructor
        Car car1 = new Car("BMW", 120);
        Car car2 = new Car("Audi", 150);

        car1.Drive();  // BMW is driving at 120 km/h
        car2.Drive();  // Audi is driving at 150 km/h
    }
}
```

Inheritance

Inheritance in C#

- When a class **includes** a **property** of another class it is known as inheritance.
- Inheritance is a process of object **reusability**.
- For example, a child includes the properties of its parents.

Syntax of **Inheritance**:

```
class BaseClass {
// Base class members
 }

 class DerivedClass : BaseClass {
 // Derived class members
 }
```

Types of Inheritance in C#

1. **Single Inheritance** – One base class and one derived class.
   BaseClass --> DerivedClass

2. **Multilevel Inheritance** – A class derived from another derived class.
   BaseClass --> BaseClass2 --> DerivedClass

3. **Hierarchical Inheritance** – Multiple classes inherit from one base class.
   BaseClass --> BaseClass2
   BaseClass --> DerivedClass

4. **Multiple Inheritance (via Interfaces only)** – C# doesn't support multiple class inheritance, but you can achieve it using **interfaces**.
   BaseClass --> DerivedClass
   BaseClass1 --> DerivedClass

Inheritance in C#

```
using System;

class Animal   // Base class
{
    public void Eat()
    {
        Console.WriteLine("Animal eats food");
    }
}

class Dog : Animal   // Derived class (inherits Animal)
{
    public void Bark()
    {
        Console.WriteLine("Dog barks");
    }
}
```

```
class Program
{
    public static void Main()
    {
        Dog dog = new Dog();

        dog.Eat();   // From Animal
        dog.Bark();  // From Dog
    }
}
```

Output:
Animal eats food
Dog barks

Polymorphism

Polymorphism in C#

- Polymorphism means one name, many forms.
- One function behaves in different forms.
- In other words, "Many forms of a single object/function is called Polymorphism.“
- Having two or more methods with same name but different in parameters, is known as **method overloading**.
- If derived class defines same method as defined in its base class, it is known as **method overriding** in C#. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the method which is already provided by its base class.

Types Polymorphism in C#

- **Compile-time Polymorphism (Static binding / Early binding)**

  Achieved by **Method Overloading** and **Operator Overloading**.

- **Run-time Polymorphism (Dynamic binding / Late binding)**

  Achieved by **Method Overriding** using `virtual`, `override`, and `abstract`.

Compile Time Polymorphism in C#

```
class Calculator
{
    // Method Overloading
    public int Add(int a, int b)
    {
        return a + b;
    }
    public double Add(double a, double b)
    {
        return a + b;
    }
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

```
class Program
{
   static void Main()
   {
     Calculator calc = new Calculator();
     Console.WriteLine(calc.Add(2, 3)); // Calls int Add
     Console.WriteLine(calc.Add(2.5, 3.5)); // Calls double Add
     Console.WriteLine(calc.Add(1, 2, 3)); // Calls 3-param Add
   }
}
```

Output:

```
5
6
6
```

Run Time Polymorphism in C#

```
class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}

class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Cat meows");
    }
}

class Program
{
    public static void Main()
    {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.Speak();   // Output: Dog barks
        a2.Speak();   // Output: Cat meows
    }
}
```

Output:
Dog barks
Cat meows

Abstraction

Abstraction in C#

- **Abstraction** is to represent the essential **feature** without representing the background details.
- Abstraction provides you a generalized view of your classes or objects by providing **relevant** information.
- Abstraction is the process of **hiding** the **working styl**e of an object and showing the information of an object in an **understandable** manner.

C# provides two main ways to achieve abstraction:

1. **Abstract Classes**
2. **Interfaces**

Abstraction in C#

- Abstract Class
- A class declared with the keyword `abstract`.
- Cannot be instantiated directly.
- Can contain **abstract methods (without body)** and **regular methods (with body)**.
- Provides **partial abstraction** (some implementation + some unimplemented methods).

Abstraction in C#

```
// Abstract class
public abstract class Shape
{
    // Abstract method (no implementation here)
    public abstract void Draw();

    // Normal method
    public void Display()
    {
        Console.WriteLine("This is a shape");
    }
}
```

```
// Derived class
public class Circle : Shape
{
    // Must provide implementation of
     abstract method
    public override void Draw()
    {
        Console.WriteLine("Drawing a
     Circle");
    }
}
```

```
//Main
class Program
{
    static void Main(string[] args)
    {
        // abstraction in action
        Shape shape = new Circle();
        shape.Draw();       // Drawing a Circle
        shape.Display();    // This is a shape
    }
}
```

Output:
Drawing a Circle
This is a shape

Encapsulation

Encapsulation in C#

- **Wrapping up** a data member and a method **together** into a single unit (in other words class) is called Encapsulation.
- Encapsulation is like enclosing in a capsule. That is **enclosing** the related **operations** and **data** related to an object into that object.
- A fully encapsulated class has **getter** and **setter** functions that are used to **read** and **write** data. This class does not allow **data access** directly.

```
class Bag {
    book;
    pen;
    ReadBook();
}
```

Encapsulation in C#

```
class Student
{
    private string name;   // private field
    private int age;       // private field

    // Public property for Name
    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    // Public property with validation for Age
    public int Age
    {
        get { return age; }
        set
        {
            if (value > 0)
                age = value;
            else
                Console.WriteLine("Age negative");
        }
    }
}
```

```
class Program
{
   public static void Main()
    {
        Student s = new Student();

        s.Name = "Rahul";
        s.Age = -5;         // Invalid, validation starts
        s.Age = 20;         // Valid

        Console.WriteLine("Name: + s.Name+ ",
Age: " + s.Age );
    }
}
```

Output:

Age negative

Name: Rahul, Age: 20

MAQ Software

Discussion



Microsoft Premier Supplier
Gold Microsoft Partner
Winner Microsoft Partner 2021 Partner of the Year Power BI Award
amazon web services Partner Network CONSULTING PARTNER
bsi. ISO/IEC 27001 Information Security Management
Inc.5000 The Fastest Growing Private Companies in America